

AI工具使用声明

本参赛队在竞赛过程中使用了 ChatGPT，主要用于建模方案讨论、代码检查与调试；参赛队已人工修改建模方案并核验代码。详见支撑文件。

附录

A 支撑文件列表

1. AI工具使用详情.pdf、AI使用说明.docx、AI使用说明.txt。
2. 源码/：全部代码文件。
3. 结果/：结果工作簿、结果汇总、逐日结果、主策略明细、调整账本、指定日期表和敏感性结果。
4. 图表/：结果图。
5. 环境/、核验/、准备复现.py、核验支撑包.py、README_支撑材料说明.txt、参考文献来源.txt、文件校验清单.json。

B 核心源码

问题一

```
def solve(p, L, V, relax=False):
    dim=6*N+1
    G, C, D, W, E, Z=0, N, 2*N, 3*N, 4*N, 5*N+1
    cost=np.zeros(dim); cost[G:G+N]=p
    lo=np.zeros(dim); hi=np.full(dim, np.inf)
    hi[C:C+N]=CAP; hi[D:D+N]=CAP
    hi[W:W+N]=V*DT
    lo[E:E+N+1]=1200; hi[E:E+N+1]=10800
    hi[Z:Z+N]=1
    a=lil_matrix((4*N+1, dim)); lower=np.full(4*N+1, -np.inf); upper=np.zeros(4*N+1)
    a[4*N, [E, E+N]]=[-1, -1]
    lower[4*N]=upper[4*N]=0
    for t in range(N):
        a[t, [G+t, C+t, D+t, W+t]]=[-1, -1, 1, -1]
        lower[t]=upper[t]=(L[t]-V[t])*DT
        a[N+t, [E+t+1, E+t, C+t, D+t]]=[-1, -1, -ETA, 1/ETA]
        lower[N+t]=upper[N+t]=0
        a[2*N+t, [C+t, Z+t]]=[-1, -CAP]
        a[3*N+t, [D+t, Z+t]]=[-1, CAP]; upper[3*N+t]=CAP
    integer=np.zeros(dim); integer[Z:]=0 if relax else 1
    r=milp(cost, integrality=integer, bounds=Bounds(lo, hi),
           constraints=LinearConstraint(a.tocsc(), lower, upper),
           options={'mip_rel_gap':1e-9, 'time_limit':120})
    assert r.success, (r.status, r.message)
    return r, [r.x[s:s+N].copy() for s in [G, C, D, W]], r.x[E:E+N+1].copy()
```

问题二

```
def action(E, r, p, future, grid=GRID):
    if r<=0:
        c=max(0, min(-r, CAP, (EMAX-E)/ETA)); return c, 0., 0., max(0, -r-c), E+ETA*c
    lo=max(EMIN, E-min(r, CAP)/ETA)
    knots=grid[(grid>lo-1e-9)&(grid<=E+1e-9)]
    knots=np.unique(np.r_[lo, E, knots]); knots=np.clip(knots, lo, E)
    costs=5*p*np.maximum(r-ETA*(E-knots), 0)+np.interp(knots, grid, future)
    nxt=float(knots[np.argmin(costs)])
    q=max(0, ETA*(E-nxt)); e=max(0, r-q)
    return 0., q, e, 0., nxt

def bellman(p, g, m, kappa, grid=GRID):
    H=len(p); V=np.empty((H+1, 3, len(grid))); V[H]=-kappa*grid
    for t in range(H-1, -1, -1):
        for s in range(3):
            future=V[t+1, s] if t==H-1 else m['P'][t, s]@V[t+1]
            if 'samples' not in m:
                V[t, s]=grid_value(future, m['net'][t, s]-g[t], p[t], grid)
            else:
                V[t, s]=0.
                for value, w in zip(m['samples'][t, s], m['weights'][t, s]):
                    if w>0: V[t, s]+=w*grid_value(future, value-g[t], p[t], grid)
    assert np.isfinite(V).all() and np.diff(V, axis=2).max()<TOL
    return V
```

问题三

```
def bellman_batch(p, plans, m, grid=GRID):
    plans=np.atleast_2d(plans);K,H=plans.shape;G=len(grid)
    V=np.zeros((K,H+1,3,G)); step=grid[1]-grid[0]
    width=int(np.floor(CAP/ETA/step+1e-10))+1
    ix=np.arange(G)[:,None]-np.arange(width)[None,:]
    valid=ix>=0;ix=np.maximum(ix,0)
    for t in range(H-1,-1,-1):
        f_all=V[:,t+1] if t==H-1 else np.einsum('ij,kjg->kig',m['P'][t],V[:,t+1])
        for s in range(3):
            f=f_all[:,s];r=m['samples'][t,s][None,:]-plans[:,t,None]
            x=grid[None,None,:]
            distance=np.minimum(np.maximum(r,0),CAP)/ETA
            lo=np.maximum(EMIN,x-distance[:, :, None])
            knot=f+5*p[t]*ETA*grid
            windows=np.where(valid[None, :, :],knot[:,ix],np.inf)
            windows=np.minimum.accumulate(windows,axis=2)
            size=np.floor(distance/step+1e-10).astype(int)
            minima=windows[np.arange(K)[ :,None,None],np.arange(G)[None,None, :],size[:, :, None]]
            boundary=interp_batch(f,lo,grid)+5*p[t]*ETA*lo
            deficit=5*p[t]*(r[:, :, None]-ETA*x)+np.minimum(minima,boundary)
            nxt=np.minimum(EMAX,x+ETA*np.minimum(np.maximum(-r,0),CAP)[ :, :, None])
            surplus=interp_batch(f,nxt,grid)
            cost=np.where(r[:, :, None]>0,deficit,surplus)
            V[:,t,s]=np.einsum('ksg,s->kg',cost,m['weights'][t,s])
    assert np.isfinite(V).all() and np.diff(V,axis=3).max()<TOL
    return V
```

问题四

```
def bellman(g, mdl, grid, scalar=False):
    h,s,l=mdl.mass.shape; assert np.asarray(g).shape==(h,)
    w=np.zeros((h+1,s,len(grid)))
    for t in range(h-1,-1,-1):
        future=mdl.transition[t]@w[t+1] if t<h-1 else w[t+1]
        if scalar:
            for state in range(s):
                for j,e in enumerate(grid):
                    w[t,state,j]=sum(mdl.mass[t,state,a]*action(e,mdl.demand[t,state,a]-g[t],
                        mdl.price[t,state,a],future[state],grid)[1] for a in range(1) if mdl.mass[
t,state,a]>0)
                continue
            gap=mdl.demand[t]-g[t]; pp=mdl.price[t]; e=grid[None,None,:]
            lower=np.maximum(LOW,e-np.minimum(np.maximum(gap,0),RATE)[ :, :, None]/ETA)
            at_lower=interpolate_rows(future,lower,grid)+5*pp[:, :, None]*(gap[:, :, None]-ETA*(e-lower))
            best=5*pp[:, :, None]*gap[:, :, None]+future[:, None, :]
            best=np.minimum(best,at_lower)
            reach=np.minimum(np.maximum(gap,0),RATE)/ETA
            for offset in range(1,min(len(grid),int(np.ceil(RATE/ETA/(grid[1]-grid[0])))+1)):
                delta=offset*(grid[1]-grid[0])
                trial=future[:,None,:-offset]+5*pp[:, :, None]*(gap[:, :, None]-ETA*delta)
                valid=reach>=delta
                best[:, :, offset]=np.minimum(best[:, :, offset:],np.where(valid[:, :, None],trial,np.inf))
            en=np.minimum(HIGH,e+ETA*np.minimum(np.maximum(-gap,0),RATE)[ :, :, None])
            surplus=interpolate_rows(future,en,grid)
            costs=np.where((gap<=0)[ :, :, None],surplus,best)
            w[t]=np.sum(mdl.mass[t, :, :, None]*costs,axis=1)
    return w
```